ZYN-32 RISC-V Core

Register File Module

zyn_basereg.v — RTL Design Documentation

Pipeline Stage: Between Stage 2 (Decode) and Stage 3 (Execute)

Language: Verilog RTL | ISA: RV32I

1. Module Overview

zyn_basereg is the **32 integer register file** for the ZYN-32 processor core. It provides **2 read ports** (for source operands rs1 and rs2) and **1 write port** (for destination operand rd). The register file implements the **RISC-V x0 = 0 rule** where register x0 is hardwired to zero and cannot be written.

1.1 Role in the 5-Stage Pipeline

The register file sits **between Decode (Stage 2) and Execute (Stage 3)**:

- Stage 2 (Decode): Sends register addresses rs1_addr, rs2_addr
- Register File: Reads values from x[rs1_addr] and x[rs2_addr]
- Stage 3 (Execute): Uses o_rs1 and o_rs2 as ALU operands
- Stage 5 (Writeback): Writes final result back via rd_addr and i_rd

1.2 Key Features

- 32 registers (x0 to x31), each 32 bits wide
- x0 hardwired to zero (reads always return 0, writes are ignored)
- Synchronous read (1-cycle latency for FPGA BRAM inference)
- Synchronous write (stable timing, matches BRAM behavior)
- Clock-enable control for power savings during stalls/bubbles

2. Port List & Signal Descriptions

Port Name	Dir	Width	Description
i_clk	in	1	System clock
i_ce_read	in	1	Clock-enable for read (from Decode stage CE)
i_rs1_addr	in	5	Source register 1 address (from Decode)
i_rs2_addr	in	5	Source register 2 address (from Decode)
i_rd_addr	in	5	Destination register address (from Writeback)
i_rd	in	32	Data to write to destination register (from Writeback)
i_wr	in	1	Write enable (from Writeback)
o_rs1	out	32	Value of register x[rs1_addr] (to Execute)
o_rs2	out	32	Value of register x[rs2_addr] (to Execute)

2.1 Internal Signals

Signal	Type	Width	Purpose
`rs1_addr_q`	reg	5	Registered copy of i_rs1_addr (for synchronous BRAM read)
`rs2_addr_q`	reg	5	Registered copy of i_rs2_addr (for synchronous BRAM read)
`base_regfile[31:1]`	reg	32x31	Register array (x1 to x31, x0 is not stored)
`write_to_basereg`	wire	1	Qualified write enable (i_wr && i_rd_addr≠0)

3. The 32 Integer Registers

RISC-V defines **32 general-purpose registers**, numbered x0 to x31. Each register is **32 bits wide** in RV32I. The registers have standard **ABI (Application Binary Interface) names** that indicate their typical usage.

3.1 Register Naming and Usage

Register	ABI Name	Typical Usage	Saved Across Calls?
x0	zero	Hardwired zero	—
x1	ra	Return address	No
x2	sp	Stack pointer	Yes
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-x7	t0-t2	Temporaries	No (caller-saved)
x8	s0/fp	Saved register / Frame pointer	Yes (callee-saved)
x9	s1	Saved register	Yes (callee-saved)
x10-x11	a0-a1	Function arguments / return values	No
x12-x17	a2-a7	Function arguments	No
x18-x27	s2-s11	Saved registers	Yes (callee-saved)
x28-x31	t3-t6	Temporaries	No (caller-saved)

3.2 The x0 = 0 Rule

x0 is always zero, regardless of what is written to it. This is a fundamental RISC-V rule enforced in hardware.

Why Hardwire x0 to Zero?

- Simplifies common operations — many instructions need to use zero as an operand
- Example: ADDI x1, x0, 10 → x1 = 0 + 10 (load immediate 10 into x1)
- Example: BEQ x1, x0, label → if (x1 == 0) branch (compare with zero)
- Saves instruction encoding space — no need for a separate 'load zero' instruction

Implementation in RTL:

```
// x0 is NOT part of the register array!
reg[31:0] base_regfile[31:1];  // Only x1 to x31

// On read: if address is 0, return 0 (not from array)
assign o_rs1 = (rs1_addr_q == 0) ? 0 : base_regfile[rs1_addr_q];
assign o_rs2 = (rs2_addr_q == 0) ? 0 : base_regfile[rs2_addr_q];
```

```
// On write: if address is 0, block the write
wire write_to_basereg = i_wr && (i_rd_addr != 0);
```

Result: x0 is never stored in the array, reads return 0, writes are ignored.

4. Synchronous Read Architecture

The register file uses **synchronous read** — addresses are **registered before reading**. This introduces a **1-cycle latency** but provides critical benefits for FPGA implementation.

4.1 Why Synchronous Read? (Not Combinational)

Option 1: Combinational Read (NOT Used)

```
// Combinational (hypothetical):
assign o_rs1 = base_regfile[i_rs1_addr];
assign o_rs2 = base_regfile[i_rs2_addr];
```

Problems with combinational read:

- **Long combinational path:** Decoder → register address extraction → regfile read → ALU input. If this path is too long, the maximum clock frequency must be lowered.
- **FPGA resource waste:** FPGA synthesis tools will use LUTs (logic cells) instead of Block RAM (BRAM). LUTs are precious resources that should be saved for logic, not storage.
- **Power consumption:** Combinational paths consume power continuously, whereas registered paths only switch on clock edges.

Option 2: Synchronous Read (USED)

```
always @(posedge i_clk) begin
    if (i_ce_read) begin
        rs1_addr_q <= i_rs1_addr;  // Register address
        rs2_addr_q <= i_rs2_addr;  // Register address
    end
end

assign o_rs1 = (rs1_addr_q == 0) ? 0 : base_regfile[rs1_addr_q];
assign o_rs2 = (rs2_addr_q == 0) ? 0 : base_regfile[rs2_addr_q];
```

Benefits of synchronous read:

- **FPGA BRAM inference:** Synthesis tools recognize this pattern and use Block RAM primitives, which are fast, dedicated memory blocks in the FPGA.
- **Shorter combinational paths:** The decoder only needs to drive the address input (not the full read datapath), improving timing closure.
- **Lower power:** Clock gating can disable reads when i_ce_read = 0 (during stalls or bubbles).

4.2 Timing Diagram — Synchronous Read

```
Instruction: ADD x3, x1, x2

Cycle N [DECODE]:       Decoder extracts rs1_addr=1, rs2_addr=2
                        Decoder drives i_rs1_addr=1, i_rs2_addr=2
                        (combinational output from decoder)

Cycle N+1 [REGFILE]:    Regfile registers addresses on posedge:
                        rs1_addr_q <= 1, rs2_addr_q <= 2
                        BRAM starts read access

Cycle N+2 [EXECUTE]:    Regfile outputs valid data:
                        o_rs1 = base_regfile[1], o_rs2 = base_regfile[2]
```

```
                    ALU receives operands and computes result
```

Key observation: There is a **1-cycle latency** between sending an address and receiving the data. This is **already accounted for** in the 5-stage pipeline design — the ALU doesn't need the data until 2 cycles after decode anyway.

5. Clock-Enable Control (i_ce_read)

The `i_ce_read` signal controls whether the register file latches new addresses. When the decode stage is **stalled** or contains a **pipeline bubble** (invalid instruction), `i_ce_read = 0`, and the addresses are **frozen**.

5.1 Why Use Clock-Enable?

- **Power savings:** Avoid updating registers when the data isn't needed (stalled or bubble cycles)
- **Glitch prevention:** Keep addresses stable during stalls to avoid spurious reads
- **Pipeline correctness:** When decode is stalled, we don't want to latch a new address — we need to keep the old one

5.2 Example Timeline

```
Cycle 1: i_ce_read=1, i_rs1_addr=5  →  rs1_addr_q <= 5 (latch new address)
Cycle 2: i_ce_read=0 (stalled)      →  rs1_addr_q remains 5 (don't change)
Cycle 3: i_ce_read=0 (stalled)      →  rs1_addr_q remains 5
Cycle 4: i_ce_read=1, i_rs1_addr=7  →  rs1_addr_q <= 7 (latch new address)
```

Without clock-enable: During stalls, random address bits might propagate and cause unintended reads.

6. Synchronous Write Architecture

Writes to the register file are **synchronous** — data is written on the **rising edge of the clock**. This matches FPGA BRAM behavior and ensures stable timing.

6.1 Write Logic

```
wire write_to_basereg = i_wr && (i_rd_addr != 0);

always @(posedge i_clk) begin
    if (write_to_basereg) begin
        base_regfile[i_rd_addr] <= i_rd;  // Synchronous write
    end
end
```

Key points:

- `write_to_basereg` is a qualified write enable — it's high only if `i_wr=1` AND `i_rd_addr≠0`
- If i_rd_addr=0, the write is blocked (x0 remains zero)
- The write occurs on the clock edge, not combinationally

6.2 Why Synchronous Write?

- **FPGA BRAM requirement:** Block RAM primitives are inherently synchronous — writes must be clocked
- **Glitch immunity:** If the write was combinational, any transient change in i_rd would corrupt the register. Synchronous writes only capture stable values on the clock edge.
- **Pipeline timing:** The writeback stage (Stage 5) computes i_rd combinationally during Cycle N, and the write happens on the clock edge entering Cycle N+1. This gives a full clock cycle for the logic to settle.

6.3 Write Timing Example

```
Instruction: ADD x3, x1, x2

Cycle 1 [FETCH]:       Fetch ADD instruction
Cycle 2 [DECODE]:      Decode, send rs1_addr=1, rs2_addr=2 to regfile
Cycle 3 [EXECUTE]:     Regfile drives o_rs1, o_rs2; ALU computes result
Cycle 4 [MEMORY]:      (no memory access for ADD)
Cycle 5 [WRITEBACK]:   Writeback drives i_rd_addr=3, i_rd=result, i_wr=1
                       (combinational signals, not yet written)

Cycle 6 [NEXT INST]:   On posedge entering Cycle 6:
                       base_regfile[3] <= result
                       (write completes)
```

Key observation: The write happens **one cycle after** the writeback stage asserts i_wr. This is expected — register file writes are **registered outputs** of the writeback stage.

7. Read-During-Write Hazard

Question: What happens if we write to x1 and read x1 **in the same clock cycle**?

7.1 The Scenario

```
Cycle N:  Writeback writes 100 to x1 (i_rd_addr=1, i_rd=100, i_wr=1)
Cycle N:  Decode reads x1 (i_rs1_addr=1, i_ce_read=1)

Question: Does Decode see the old value or the new value (100)?
```

7.2 Answer: You Get the OLD Value

Both operations happen on the **same clock edge**:

- The write updates the register file array
- The read latches the address into rs1_addr_q
- But the read sees the array BEFORE the write updates it (Verilog semantics)

```
always @(posedge i_clk) begin
    // Both execute simultaneously on the clock edge
    if (write_to_basereg)
        base_regfile[i_rd_addr] <= i_rd;  // Updates array
    if (i_ce_read)
        rs1_addr_q <= i_rs1_addr;         // Latches address
end

// Next cycle:
assign o_rs1 = base_regfile[rs1_addr_q];  // Reads old value!
```

7.3 Solution: Operand Forwarding Unit

The register file is **NOT responsible** for solving this hazard. Instead, the **zyn_forwarding module** detects the conflict and **forwards the new value** directly from the writeback stage to the ALU, bypassing the register file.

```
// In zyn_forwarding:
if ((decoder_rs1_addr == writeback_rd_addr) && writeback_wr_rd && writeback_ce)
    o_rs1 = writeback_rd;  // Forward from writeback
else
    o_rs1 = rs1_orig;      // Use regfile value
```

Design lesson: The register file is a simple storage element. **Hazard resolution** is handled by a separate forwarding unit.

8. Annotated RTL Code

```
// ═══════════════════════════════════════════════════════════════
//  zyn_basereg.v  —  32 Integer Register File
//  ZYN-32 RISC-V Core
// ═══════════════════════════════════════════════════════════════
`timescale 1ns / 1ps
`default_nettype none

module zyn_basereg (
    input  wire        i_clk,
    input  wire        i_ce_read,      // Clock-enable for read
    input  wire [4:0]  i_rs1_addr,     // Source register 1 address
    input  wire [4:0]  i_rs2_addr,     // Source register 2 address
    input  wire [4:0]  i_rd_addr,      // Destination register address
    input  wire [31:0] i_rd,           // Data to write
    input  wire        i_wr,           // Write enable
    output wire [31:0] o_rs1,          // Source register 1 value
    output wire [31:0] o_rs2           // Source register 2 value
);

    // ── Internal Signals ──────────────────────────────────────
    reg [4:0]  rs1_addr_q, rs2_addr_q;  // Registered addresses
    reg [31:0] base_regfile[31:1];      // x1 to x31 (x0 not stored)
    wire write_to_basereg;

    // ── Synchronous Read/Write ────────────────────────────────
    always @(posedge i_clk) begin
        // Synchronous write (if enabled and not writing to x0)
        if (write_to_basereg) begin
            base_regfile[i_rd_addr] <= i_rd;
        end

        // Synchronous read (register addresses when enabled)
        if (i_ce_read) begin
            rs1_addr_q <= i_rs1_addr;
            rs2_addr_q <= i_rs2_addr;
        end
    end

    // ── Write Enable (block writes to x0) ─────────────────────
    assign write_to_basereg = i_wr && (i_rd_addr != 0);

    // ── Read Outputs (x0 hardwired to zero) ───────────────────
    assign o_rs1 = (rs1_addr_q == 0) ? 0 : base_regfile[rs1_addr_q];
    assign o_rs2 = (rs2_addr_q == 0) ? 0 : base_regfile[rs2_addr_q];

endmodule
```

9. Design Lessons from zyn_basereg

9.1 Synchronous Reads for FPGA BRAM Inference

By registering addresses before reading, the register file **maps directly to FPGA Block RAM primitives**. This saves LUTs and improves performance.

9.2 x0 = 0 Rule Simplifies Hardware

Hardwiring x0 to zero eliminates the need for **'load immediate zero'** instructions and **'compare with zero'** special cases. The cost (one less usable register) is negligible.

9.3 Clock-Enable for Power and Correctness

Using `i_ce_read` to gate register updates **saves power** during stalls and **prevents glitches** from invalid addresses.

9.4 Separation of Concerns: Hazards ≠ Register File

Read-during-write hazards are **not solved** in the register file. A separate **forwarding unit** handles hazard detection and resolution. This keeps the register file simple and synthesizable.

9.5 1-Cycle Latency is Acceptable

The synchronous read introduces **1-cycle latency**, but the 5-stage pipeline **already accounts for this**. The ALU doesn't need the data until 2 cycles after decode anyway.

10. Glossary

Term	Definition
Register File	Fast storage array inside the CPU for holding intermediate values during computation.
x0 = 0 Rule	RISC-V hardware rule: register x0 always reads as zero, writes to x0 are ignored.
Synchronous Read	Addresses are registered before reading (1-cycle latency). Required for FPGA BRAM inference.
Synchronous Write	Data is written on the clock edge, not combinationally. Matches BRAM behavior.
BRAM (Block RAM)	Dedicated memory blocks in FPGAs. Fast, efficient, but require synchronous access.
LUT (Look-Up Table)	FPGA logic cells. Used for combinational logic. Should not be wasted on storage.
Clock-Enable	Signal that controls whether a register updates. Used for power savings and glitch prevention.
Read-During-Write Hazard	Conflict when reading and writing the same register in the same cycle. Solved by forwarding unit.
ABI (Application Binary Interface)	Standard naming and usage conventions for registers (e.g., x1 = ra, x2 = sp).

11. References

- RISC-V Instruction Set Manual, Volume I: User-Level ISA (Chapter 2: RV32I Base Integer Instruction Set)
- RISC-V Unprivileged Specification (Register File, x0 = 0 rule)
- RISC-V ABI Specification (Register naming conventions: zero, ra, sp, gp, tp, t0-t6, s0-s11, a0-a7)
- Angelo Jacobo's RISC-V Implementation: https://github.com/AngeloJacobo/RISC-V